

Nome do Software: App_Scholar

Nome do(a) Aluno(a): João Pedro Souza dos Anjos

1. Introdução

1.1. Objetivo do Projeto

Desenvolver um sistema de gestão acadêmica móvel que possibilite aos alunos e professores o acesso centralizado e intuitivo a disciplinas, notas, frequências e resumo do desempenho acadêmico (boletim).

1.2. Escopo do Sistema

O sistema é um aplicativo mobile focado na plataforma **Android**, permitindo que o usuário consulte seu histórico escolar e acompanhe o progresso semestral de forma remota.

1.3. Público-Alvo

Discentes (Alunos), Docentes (Professores) e Administradores da instituição de ensino.

1.4. Siglas e/ou Abreviações

- **API:** Interface de Programação de Aplicações.
- **CRUD:** Create (Criar), Read (Ler), Update (Atualizar), Delete (Deletar).
- **UI/UX:** Interface e Experiência do Usuário.
- **JWT:** JSON Web Token (protocolo de autenticação).

2. Visão Geral do Sistema

2.1. Funcionalidades Principais

- Autenticação segura via Login (Token JWT).
- Visualização de disciplinas matriculadas com carga horária e semestre.
- Visualização de notas (N1, N2) e média final.
- Resumo estatístico do desempenho (aprovados, reprovados, média geral).

2.2. Plataformas Suportadas

- Android

3. Requisitos do Sistema

3.1. Requisitos Funcionais

- **RF001:** O sistema deve validar login e senha dos usuários.
- **RF002:** O sistema deve listar as disciplinas vinculadas ao aluno.
- **RF003:** O sistema deve calcular a média aritmética e situacional (aprovado/reprovado).
- **RF004:** O sistema deve sincronizar os dados em tempo real com o servidor remoto.

3.2. Requisitos Não Funcionais

- **RNF001:** O tempo de resposta para carregamento de boletim deve ser inferior a 3 segundos.
- **RNF002:** Interface responsiva adaptada para diferentes resoluções de tela de smartphones.
- **RNF003:** Comunicação segura entre App e Servidor via HTTPS.

4. Arquitetura do Sistema

4.1. Tecnologias Utilizadas

- **Linguagem:** JavaScript / TypeScript.
- **Framework:** React Native (Expo).
- **Banco de Dados:** PostgreSQL.
- **Back-end:** Node.js (Express).
- **API:** RESTful.

4.2. Camadas da Arquitetura

- **Apresentação:** Telas desenvolvidas em React Native.
- **Serviço:** Requisições HTTP (Axios) para o Back-end.
- **Persistência:** Banco de dados relacional remoto e armazenamento local de sessão (AsyncStorage).

5. Modelagem de Dados

5.1. Diagrama de Entidade-Relacionamento (DER)

Alunos: id (PK), nome, matricula, curso, email.

Disciplinas: id (PK), nome, carga_horaria, semestre, professor_id (FK).

Notas: id (PK), aluno_id (FK), disciplina_id (FK), nota1, nota2, media, situacao.

Professores: id (PK), nome, titulacao.

5.2. Estrutura de Tabelas (exemplo)

Tabela: usuários

Tabela: Alunos

- **id:** Inteiro, Chave Primária (PK), Auto-incremento.
- **nome:** Varchar(100), Obrigatório.
- **matricula:** Varchar(20), Único, Obrigatório.
- **curso:** Varchar(50), Obrigatório.
- **email:** Varchar(100), Único.

Tabela: Professores

- **id:** Inteiro, Chave Primária (PK).
- **nome:** Varchar(100), Obrigatório.
- **titulacao:** Varchar(50).
- **area:** Varchar(50).

Tabela: Disciplinas

- **id:** Inteiro, Chave Primária (PK).
- **nome:** Varchar(100), Obrigatório.
- **carga_horaria:** Inteiro.
- **semestre:** Inteiro.
- **curso:** Varchar(50).
- **professor_id:** Inteiro, Chave Estrangeira (FK) referenciando **Professores(id)**.

Tabela: Notas

- **id:** Inteiro, Chave Primária (PK).
- **aluno_id:** Inteiro, Chave Estrangeira (FK) referenciando **Alunos(id)**.
- **disciplina_id:** Inteiro, Chave Estrangeira (FK) referenciando **Disciplinas(id)**.
- **nota1:** Decimal(4,2).
- **nota2:** Decimal(4,2).
- **media:** Decimal(4,2).
- **situacao:** Varchar(20) (Ex: Aprovado, Reprovado, Cursando).

6. Fluxos de Navegação

6.1. Telas e Componentes

- Tela de Login.
- Dashboard (Tela Inicial).
- Tela "Minhas Disciplinas".
- Tela "Boletim Acadêmico".
- Tela "Notas".
- Tela "Cadastro Aluno".
- Tela "Cadastro Professor".
- Tela "Cadastro Disciplina".

6.2. Diagrama de Navegação

(inserir ou descrever fluxo entre as telas)

7. Segurança

7.1. Autenticação e Autorização

Uso de tokens JWT gerados no login e validados em cada requisição através de middleware no back-end.

7.2. Armazenamento Seguro

Implementação de tratamento de erros, sanitização de dados no banco (SQL Injection prevention) e criptografia de senhas (bcrypt).

8. Testes

8.1. Tipos de Testes Aplicados

- Testes Unitários
- Testes de Integração
- Testes de Interface
- Testes de Performance
- Testes de Usabilidade

8.2. Ferramenta de Teste (Sugerida)

- Debugger do Expo e Logs do terminal (Node.js).

9. Implantação

9.1. Ambientes

Back-end hospedado em serviços de nuvem (ex: Render/Railway) e banco de dados em nuvem.

9.2. Manutenção

- Atualização de bibliotecas de dependência (npm/yarn) e monitoramento de logs de erro via servidor.

10. Manutenção e Atualizações

- Atualizações de segurança periódicas
- Correções de bugs reportados via feedback ou análise de logs
- Inclusão de novas funcionalidades conforme demanda do cliente

11. Anexos

- Prints das telas
- Mockups e protótipos
- Documentos adicionais (contratos, propostas, atas de reunião, etc.)